

Deep learning / apprentissage profond

Cours 2 : notions de base

Hervé Le Borgne
herve.le-borgne@cea.fr

2019-2020

Rappels (→ pré-requis)

- historique : le deep learning est une sous partie de l'apprentissage, lui-même un sous domaine de l'intelligence artificielle.
- apprentissage supervisé : principes et détails
- le neurone formel et le perceptron
- les applications concernent la vision, l'audio, la parole, le traitement automatique de la langue, la décision, etc.
- principaux *framework* de deep learning : PyTorch et Tensorflow
 - ▶ au moins un est installé sur votre machine
- il vaut mieux avoir un (bon) GPU pour apprendre des modèles profonds

Projet

Et bien entendu vous avez commencé à réfléchir à votre **projet**

Plan

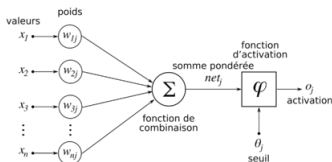
- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion

Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion

● Perceptron :

- ▶ NN 1 couche et 1 sortie = séparateur linéaire 2 classes
- ▶ procédures d'apprentissage par descente de gradient (stochastique)
- ▶ théorème assure convergence **si** données linéairement séparables
- ▶ généralisation : x_i et w_i dans $\mathbb{R}^n$ et ϕ non linéaire



$$o_j = \varphi \left(\sum_{i=1}^n w_{ij} x_i + \theta_j \right) \quad (1)$$

Objectif

Nous souhaitons approximer une fonction f^* quelconque (e.g non linéaire), par exemple un classifieur $y = f^*(\mathbf{x})$.

Un MLP définit un mapping $y = f(\mathbf{x}; \theta)$ et apprend θ pour approximer au mieux f^* .

Du perceptron au MLP

- Perceptron multi-couche

- ▶ Réseau neuronal (profond) direct : empilement de plusieurs (couches de) perceptrons
 - ★ Deep feedforward networks
 - ★ feedforward neural networks
 - ★ multi-layer perceptron (MLP)
- ▶ Couches « complètement connectées » (*fully connected*) entre elles
- ▶ Séparation non linéaire (des classes) possible
 - ★ grâce à une « combinaison de $\varphi \left(\sum_{j=1}^n w_{ij}x_j + \theta_j \right)$ »
- ▶ Apprentissage par descente de gradient (stochastique)
- ▶ Un théorème assurant la convergence... Reste à trouver !

Apprentissage MLP

L'apprentissage d'un MLP (i.e le mapping $y = f(\mathbf{x}; \theta)$) consiste à apprendre les poids $\theta = [w_1, \dots, w_i, \dots]$ associés à toutes les connexions du réseau

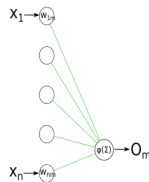
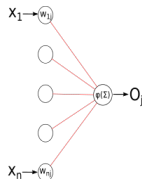
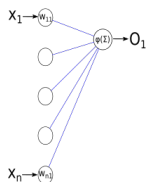
Du perceptron au MLP

- accumulation de perceptrons en couches (*layers*)

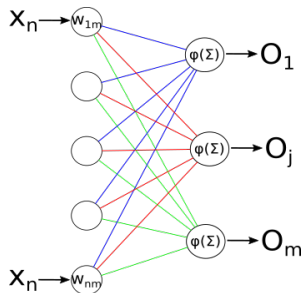
▶ perceptron = neurone formel apprenable

(oui, le mot « apprenable » existe)

(oui, le mot est assez laid...)



- les sorties de la couche forment une entrée multidimensionnelle
→ entrée couche suivante...
- $n \times m$ poids (et m biais) à apprendre.



MLP : vocabulaire

- Réseau

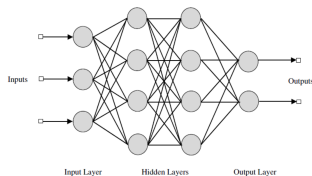
- ▶ Composition de fonction : $f(\mathbf{x}) = f_3(f_2(f_1(\mathbf{x})))$
- ▶ profondeur (*depth*) : nombre de couche

- Neuronal

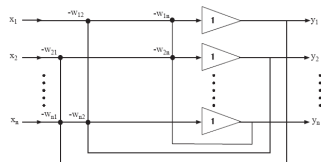
- ▶ Inspiration neuromimétique (cf. neurone formel)

- Direct ($\neq$ Récursif)

- ▶ sans retour d'information (*feedback*)

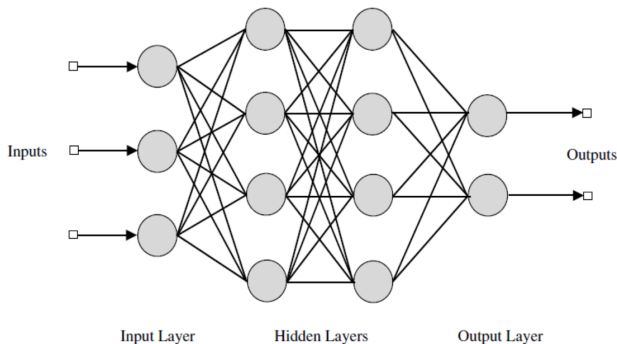


Direct



Récursif [Hérault & Jutten, 1991]

MLP : vocabulaire

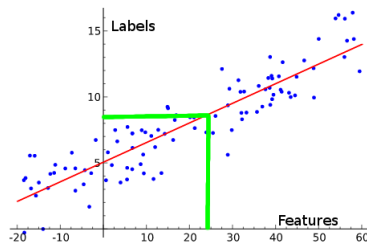


- couche d'entrée / de sortie / cachée (*hidden*)
- couche d'entrée usuellement **non** comptabilisée dans le nombre de couches (on s'intéresse aux *poids à apprendre*, pas aux entrées/sorties)
- on apprend aussi les **biais** de chaque neurone (non représentés)

Modèles linéaires

Régression :

- Apprentissage (feature 1D)
 - ▶ n données étiquetées
 - ▶ « résumées » par 2 paramètres
- Test
 - ▶ toute donnée peut être étiquetée



- La *corrélation* donne la performance d'une droite (hyperplan) à décrire les données (c'est la part de la variance des données expliquée linéairement)
- La régression en dim. d « résume » les données par $d + 1$ paramètres
- Il existe des algorithmes efficaces pour calculer les paramètres

Classification :

- Idem avec une fonction logistique (regression logistique)

credit : Science4all



Modèles linéaires : limitations

Les modèles tels que la régression linéaire ou logistique :

- + modélisent facilement des problèmes
- + leur optimisation (calcul) est aisé
- ont une capacité de modélisation limitée !
- Les étiquettes n'ont pas forcément une dépendance linéaires aux caractéristiques
- Le cas du XOR a failli sonner le glas de l'approche connexioniste !
 - ▶ Livre [Minski et Papert, *Perceptron*, 1969]

- X_1 : pose
- X_2 : couleur



Kernel trick

Astuce du noyau (*kernel trick*)

Déterminer une relation linéaire non pas sur les features $\mathbf{x}$ mais sur une transformation non linéaire $\phi(\mathbf{x})$ de ceux-ci. On parle de *mapping* non linéaire.

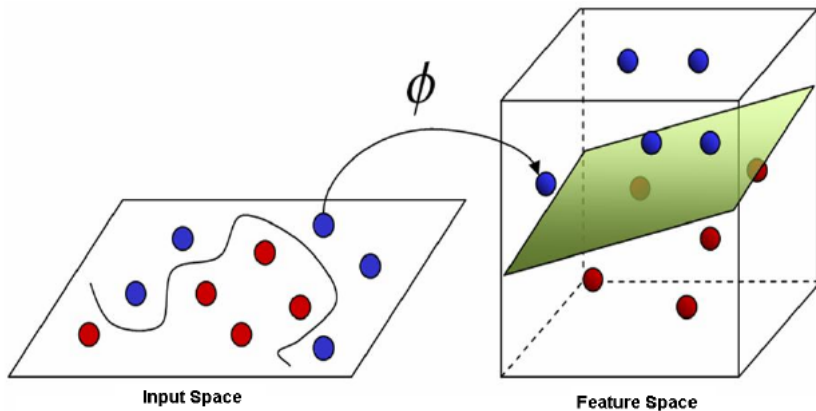
Intéressant pour des algorithmes s'exprimant en termes de produits scalaires, car sous certaines conditions (théorème de Mercer), on a :

$$\langle \phi(x_1), \phi(x_2) \rangle = K(x_1, x_2) \quad (2)$$

où K est une fonction noyau (*kernel*) évitant le calcul explicite du mapping.

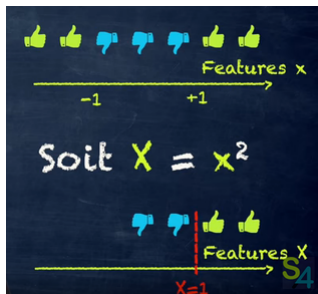
Kernel trick

- Une projection des données en plus grande dimension peut permettre d'en trouver plus facilement un séparateur linéaire
 - ▶ voir aussi : fléau de la dimension (*curse of dimensionality*)

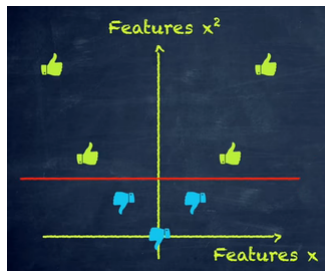


Kernel trick

- un mapping (non linéaire) des données peut permettre de les séparer **linéairement**
 - ▶ revient à « les projeter en plus grande dimension »
- Exemple en 1D :



credit : Science4all



Quel mapping non linéaire ?

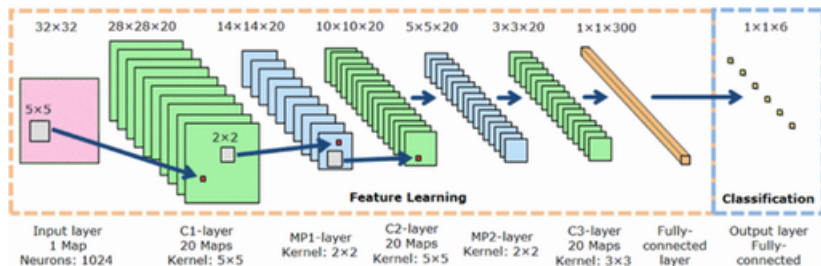
- > induit par un noyau « classique » : polynomial, gaussien (RBF), ...
 - + mise en place peu coûteuse, e.g $RBF(x_1, x_2) = e^{-\frac{\|x_1 - x_2\|^2}{\sigma^2}}$
 - trop générique pour des problèmes particuliers
- > conçu « à la main »
 - + codage connaissance *a priori* d'un problème particulier dans le noyau
 - très coûteux à mettre en place
 - peu de transfert d'un domaine à un autre (TAL, speech, vision...)
- > en *deep learning* on... Apprend le mapping !

$$y = f(\mathbf{x}; \theta, \mathbf{w}) = \phi(\mathbf{x}; \theta)^T \mathbf{w} \quad (3)$$

- ▶ déterminer θ = apprendre le mapping ϕ (e.g couches cachées MLP)
- ▶ paramètres $\mathbf{w}$ permettent le mapping de $\phi(\mathbf{x})$ vers le label désiré
- + on peut utiliser des *familles* de ϕ très génériques et non le ϕ exact
- + on peut coder une connaissance *a priori* dans ϕ
 - il faut apprendre → optimisation

Apprentissage mapping non linéaire

- Les données sont transformées pour en donner une nouvelle représentation (apprentissage du noyau non linéaire !)
 - feature learning*
- On calcule un séparateur linéaire sur cette nouvelle représentations
 - classification*



(l'image ci-dessus illustre un CNN (Cf. cours 3) mais c'est pareil pour un MLP)

Exemple : apprendre XOR

XOR (OU exclusif) est défini par la fonction $y = f^*(x = [x_1, x_2]^T)$:

	0	1
0	0	1
1	1	0

On souhaite apprendre un modèle $y = f(x; \theta)$ et régler les paramètres θ de manière à approcher au mieux f^* .

Pour simplifier, nous visons simplement à nous adapter à l'ensemble d'apprentissage (pas de généralisation) :

$$\mathbb{X} = \{[0, 0]^T, [0, 1]^T, [1, 0]^T, [1, 1]^T\} \quad (4)$$

Approche régression linéaire

On choisit un coût quadratique (MSE : mean square error) :

$$J(\theta) = \frac{1}{4} \sum_{x \in \mathbb{X}} (f^*(x) - f(x; \theta))^2 \quad (5)$$

Prenons un modèle linéaire $\theta = [w_1, w_2, b]^T$:

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b \quad (6)$$

La minimisation de $J(\theta)$ mène à un problème sur-dimensionné. Une pseudo-solution est obtenu par les équations normales $(X^T X)\theta = X^T Y$:

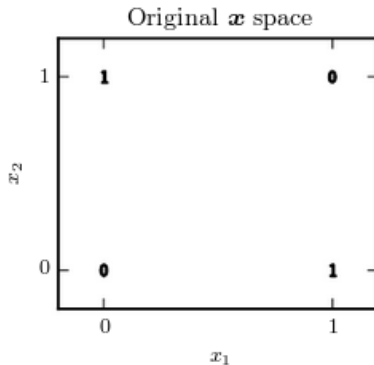
$$\theta^* = [0, 0, \frac{1}{2}]^T \quad (7)$$

X a des entrées « complétées » $[x_1 \ x_2 \ 1]$ soit : $X = [0 \ 0 \ 1; 0 \ 1 \ 1; 1 \ 0 \ 1; 1 \ 1 \ 1]$ et $Y = [0; 1; 1; 0]$

Approche régression linéaire

$$\theta^* = [0, 0, \frac{1}{2}]^T \quad (8)$$

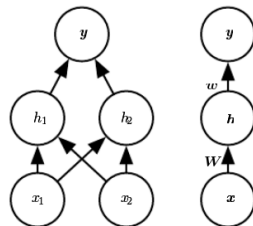
Un modèle linéaire ne peut pas résoudre XOR :



XOR NN

Réseau de neurone à une couche cachée :

- couche cachée h : fonction $f^{(1)}(\mathbf{x}; \mathbf{W}, \mathbf{c})$
- couche sortie : régression linéaire $f^{(2)}(\mathbf{h}; \mathbf{w}, b)$
 → appliquée à h et non x !



Modèle complet :

$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = f^{(2)}\left(f^{(1)}(\mathbf{x})\right) \quad (9)$$

NB :

- $\mathbf{W}$ = poids pour $f^{(1)}$ (non linéaire)
- $\mathbf{w}$ = poids pour $f^{(2)}$ (linéaire)

XOR NN

Quelle forme pour $f^{(1)}(.)$?

- Si $f^{(1)}(.)$ est linéaire alors f est globalement linéaire
- Intervention d'une fonction d'activation $g(.)$ non linéaire :

$$\mathbf{h} = g(\mathbf{W}^T \mathbf{x} + \mathbf{c}) \quad (10)$$

On utilise par exemple ReLU (*rectified linear unit*) : $g(z) = \max\{0, z\}$.

Ainsi :

$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^T \max\{0, \mathbf{W}^T \mathbf{x} + \mathbf{c}\} + b \quad (11)$$

XOR NN

Soit la solution suivante :

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$b = 0$$

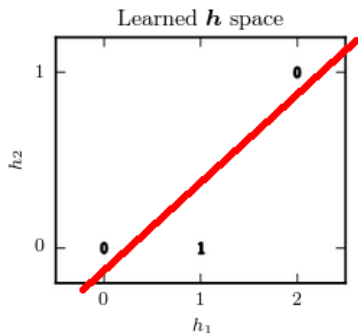
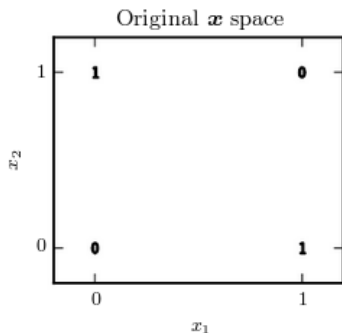
Appliquons $f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^T \max\{0, \mathbf{W}^T \mathbf{x} + \mathbf{c}\} + b$ aux entrées :

$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \quad \text{i.e pour } \mathbf{x} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{puis } \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \dots$$

XOR NN

On obtient :

$$f(\mathbf{X}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = [1; -2] \times \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \text{ pour } \mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$



Conclusion sur l'exemple XOR

- Importance d'introduire une non linéarité
 - ▶ l'espace $\mathbf{h}$ résulte d'un mapping non-linéaire des données $\mathbf{x}$
 - ▶ l'espace $\mathbf{x}$ n'est pas linéairement séparable
 - ▶ l'espace $\mathbf{h}$ est linéairement séparable
- La solution a ici été *devinée* pour $\{W, c, w, b\}$
- En pratique, on peut avoir des millions de paramètres
→ apprentissage par **optimisation**

Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation**
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion

Position du problème

Définition

Optimiser consiste à trouver un minima ou maxima (global) d'une fonction $f(\theta)$

- Maximiser $f(\theta)$ est équivalent à minimiser $-f(\theta)$
- La fonction à optimiser (minimiser) est appelée :
 - ▶ fonction de coût (*cost function*, *loss function*)
 - ▶ fonction d'erreur (*error function*)
- L'optimal est noté avec une étoile (*) :

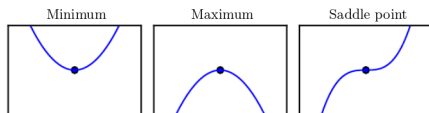
$$\theta^* = \arg \min f(\theta)$$

Descente de gradient : cas mono-dimensionnel

En 1D, la dérivée de $f(\cdot)$ donne la direction dans laquelle se déplacer pour minimiser (opposée au signe de $f'(\cdot)$ car $f(\theta + \epsilon) \approx f(\theta) + \epsilon f'(\theta)$) :

$$\exists \epsilon > 0, f(\theta - \epsilon \text{sign}(f'(\theta))) \leq f(\theta) \quad (12)$$

Si $f'(\theta) = 0$ nous sommes à un extremum ou un point selle :

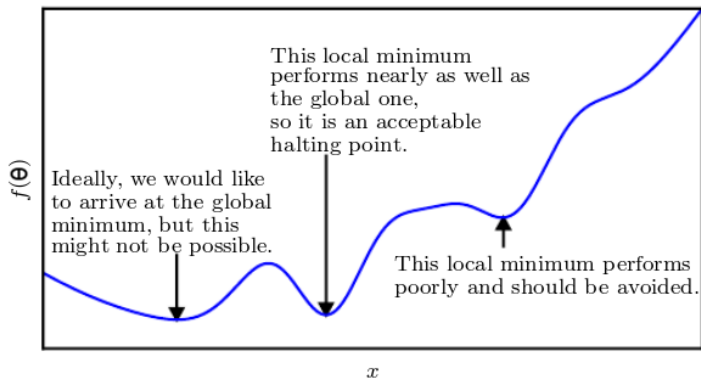


Le signe de la dérivée seconde donne la courbure :

- $f''(\theta) > 0$: courbure positive $\rightarrow$ minimum
- $f''(\theta) < 0$: courbure négative $\rightarrow$ maximum
- $f''(\theta) = 0$: point selle ou région « plate ».

Minima globaux et locaux

Les fonctions de coût en deep learning ont de nombreux minima locaux (et de points selle). Ils vau^x mieux parfois garder un « bon » minima local plutôt que de se perdre à chercher l'optimal global.



Descente de gradient

- On optimise généralement des fonctions à entrée multidimensionnelle :

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

- On optimise selon toutes les directions $\rightarrow$ dérivée partielle
- Dérivée $\rightarrow$ gradient :

$$\nabla_{\theta} f(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_1} f(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_i} f(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} f(\theta) \end{bmatrix}$$

En multiples dimensions, les points critiques ont un gradient nul.

Descente de gradient

- La dérivée directionnelle dans la direction $\mathbf{u}$ (unitaire) est la pente de f dans cette direction
- C'est donc la dérivée de $f(\boldsymbol{\theta} + \alpha \mathbf{u})$ par rapport à α (évaluée en $\alpha = 0$)
- Avec la dérivée de fonctions composées (*chain rule*), en $\alpha = 0$:

$$\frac{\partial}{\partial \alpha} f(\boldsymbol{\theta} + \alpha \mathbf{u}) = \mathbf{u}^T \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \quad (13)$$

- La direction $\mathbf{u}$ permettant de faire décroître f au plus vite est :

$$\min_{\mathbf{u}, \mathbf{u}^T \mathbf{u} = 1} \mathbf{u}^T \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = \min_{\mathbf{u}, \mathbf{u}^T \mathbf{u} = 1} \|\mathbf{u}\|_2 \|\nabla_{\mathbf{x}} f(\boldsymbol{\theta})\| \cos \beta \quad (14)$$

$$= \min_{\mathbf{u}, \mathbf{u}^T \mathbf{u} = 1} \cos \beta \quad (15)$$

Où β est l'angle entre $\mathbf{u}$ et le gradient. Minimal pour $\beta = \pi$

Descente de gradient

La direction $\mathbf{u}$ permettant de faire décroître f au plus vite est donc opposée à celle du gradient de f .

Descente de gradient (plus grande pente)

[Cauchy, 1847] Une fonction de coût $f()$ peut être itérativement minimisée en se déplaçant dans la direction opposée au gradient :

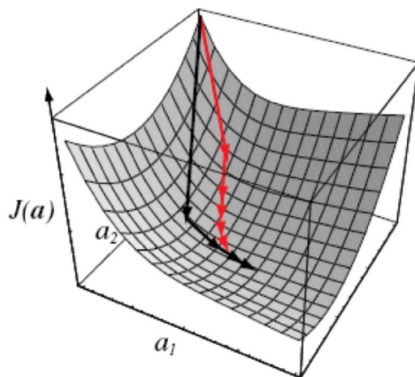
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \epsilon \nabla_{\mathbf{x}} f(\boldsymbol{\theta}) \quad (16)$$

Où $\epsilon > 0$ est le taux d'apprentissage (*learning rate*). On arrête les itérations quand $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \approx 0$

L'aspect itératif peut être parfois évité en résolvant $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = 0$

Descente de gradient

- Processus itératif de recherche du minimum de la fonction de coût J
 - ▶ Espace à deux paramètres a_1 et a_2
 - ▶ On cherche des états (a_1, a_2) donnant un coût J plus faible en « suivant la pente »
 - ▶ À chaque état, on bouge d'un pas qui est le taux d'apprentissage



Matrice jacobienne

Si la sortie est multidimensionnelle, les « gradients pour chaque composante de sortie » sont donnés par la :

Matrice jacobienne

Si $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$, sa matrice jacobienne $\mathbf{J}_f \in \mathbb{R}^{n \times m}$ est définie par :

$$\mathbf{J}_f(i, j) = \frac{\partial}{\partial \theta_j} f(\boldsymbol{\theta})_i \quad (17)$$

La composée $f \circ g$ vérifie : $\mathbf{J}_{f \circ g} = (\mathbf{J}_f \circ g) \mathbf{J}_g$

Exemple :

$$F(\theta_1, \theta_2, \theta_3) = (\theta_1, 5\theta_3, 4\theta_2^2 - 2\theta_3, \theta_3 \sin(\theta_1))$$

$$\mathbf{J}_F(\theta_1, \theta_2, \theta_3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 5 \\ 0 & 8\theta_2 & -2 \\ \theta_3 \cos(\theta_1) & 0 & \sin(\theta_1) \end{pmatrix}$$

Matrice hessienne

Matrice hessienne

Pour $f : \mathbb{R}^m \rightarrow \mathbb{R}$, l'ensemble de ses dérivées secondes forment sa matrice hessienne, définie par :

$$\mathbf{H}(f)(\boldsymbol{\theta})_{i,j} = \frac{\partial^2}{\partial \theta_i \partial \theta_j} f(\boldsymbol{\theta}) \quad (18)$$

- La hessienne de f est la matrice jacobienne du gradient de f (transposée)
- Pour $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$, la hessienne devient un tenseur d'ordre trois $H(f) = (H(f_1), \dots, H(f_n))$
- Si les dérivées partielles sont continues, $\mathbf{H}_{i,j} = \mathbf{H}_{j,i}$
- On la note aussi $\nabla_{\boldsymbol{\theta}}^2 f$

Résumé

$f : \mathbb{R} \rightarrow \mathbb{R}$	$f : \mathbb{R}^n \rightarrow \mathbb{R}$	$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
dérivée première f'	gradient $\nabla_{\theta} f$	matrice jacobienne J_f
dérivée seconde f''	matrice hessienne $H = \nabla_{\theta}^2 f = J_{\nabla_{\theta} f}$	tenseur hessien $\{H(f_1), \dots, H(f_m)\}$

Matrices hessiennes

- En deep learning, les hessiennes sont généralement réelles et symétriques
→ H est diagonalisable : base de vecteurs propres $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$
- La dérivée seconde de f dans la direction (unitaire) $\mathbf{u}$ est $\mathbf{u}^T H \mathbf{u}$
- Si $\mathbf{u}$ est un vecteur propre, la dérivée seconde est la valeur propre correspondante
- Sinon, $\mathbf{u}$ se décompose sur la base de vecteurs propres, avec $u_i \in [0, 1]$
 - > u_i est maximum pour le vecteur propre le plus proche (angle)
- $\Rightarrow$ La valeur propre donne la courbure (dérivée seconde) dans la direction du vecteur propre correspondant.

Choix taux d'apprentissage

La descente de gradient a amené le système en $\theta^{(0)}$.

- le gradient donne la direction du prochain pas
- on souhaite déterminer sa « longueur » (i.e taux d'apprentissage ϵ)

Notons $\mathbf{g} = \nabla_{\theta} f(\theta)$ le gradient de f . Le développement de Taylor au second ordre de f autour de $\theta^{(0)}$ donne :

$$f(\theta) \approx f(\theta^{(0)}) + (\theta - \theta^{(0)})^T \mathbf{g} + \frac{1}{2}(\theta - \theta^{(0)})^T \mathbf{H}(\theta - \theta^{(0)}) \quad (19)$$

Une étape de descente de gradient conduit au point $\theta = \theta^{(0)} - \epsilon \mathbf{g}$, donc :

$$f(\theta^{(0)} - \epsilon \mathbf{g}) \approx f(\theta^{(0)}) - \epsilon \mathbf{g}^T \mathbf{g} + \frac{1}{2} \epsilon^2 \mathbf{g}^T \mathbf{H} \mathbf{g} \quad (20)$$

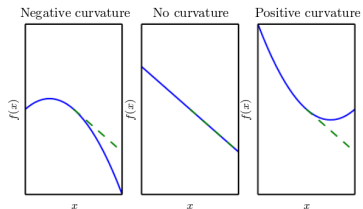
Choix taux d'apprentissage

- On souhaite déterminer le taux d'apprentissage ϵ

$$f(\theta^{(0)} - \epsilon g) \approx f(\theta^{(0)}) - \epsilon g^T g + \frac{1}{2} \epsilon^2 g^T H g \quad (21)$$

Trois termes :

- point original
- décroissance (du coût) possible en suivant la pente tangente à f
- correction à appliquer du fait de la courbure



→ courbure ≤ 0 versus courbure > 0

Choix taux d'apprentissage

- On souhaite déterminer le taux d'apprentissage ϵ

$$f(\theta^{(0)} - \epsilon g) \approx f(\theta^{(0)}) - \epsilon g^T g + \frac{1}{2} \epsilon^2 g^T H g \quad (22)$$

- Si $g^T H g \leq 0$, alors ϵ peut être aussi grand qu'on veut (dans les limites de la validité de l'approximation de Taylor!!!)
- Si $g^T H g > 0$, alors le pas optimal est (minimisation polynôme ordre 2 en ϵ) :

$$\epsilon^* = \frac{g^T g}{g^T H g} \quad (23)$$

- Si g est aligné avec un vecteur propre de H , $\epsilon^* = \frac{1}{\text{val. propre}}$
- Plus généralement, les valeurs propres de H déterminent l'échelle de ϵ

Test de la dérivée seconde généralisé

Rappel, en 1D ($f : \mathbb{R} \rightarrow \mathbb{R}$) :

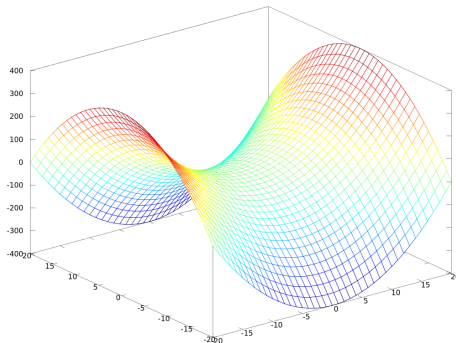
- θ^* minimum $\iff f'(\theta^*) = 0$ et $f''(\theta^*) > 0$
- θ^* maximum $\iff f'(\theta^*) = 0$ et $f''(\theta^*) < 0$
- Si $f'(\theta^*) = f''(\theta^*) = 0$ on ne peut pas conclure

On peut généraliser à plusieurs dimensions, avec les valeurs propres λ_i de la hessienne en un point critique (i.e $\nabla_x f(\theta^*) = 0$) :

- Si $\forall i, \lambda_i > 0$ (**H** est définie positive) alors θ^* est un minimum local car la dérivée seconde directionnelle est > 0 dans toutes les directions $\rightarrow$ cas 1D
- Si $\forall i, \lambda_i < 0$ alors θ^* est un maximum local
- Si $\exists i; \lambda_i = 0$ et $\lambda_{j \neq i}$ de même signe, on ne peut pas conclure

Test de la dérivée seconde généralisé

- Si $\exists i, j$ t.q $\lambda_i > 0$ et $\lambda_j < 0$, on a un point selle
 - ▶ θ^* est minimum local pour la direction e_i
 - ▶ θ^* est maximum local pour la direction e_j



Gradient stochastique (intro)

Rappel : pour une fonction de coût $f(\theta)$, la descente de gradient itère :

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} f(\theta)$$

Pour une fonction de coût classique telle la log-vraisemblance négative :

$$f(\theta) = \frac{1}{m} \sum_{i=1}^m L(x^{(i)}, y^{(i)}, \theta) \quad (24)$$

Le calcul du gradient est :

$$\nabla_{\theta} f(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(x^{(i)}, y^{(i)}, \theta) \quad (25)$$

→ complexité calculatoire $O(m)$ avec $m \sim 10^8$ par exemple

Gradient stochastique (intro)

Le gradient stochastique (ou « en ligne ») estime le gradient avec un seul exemple puis itère sur l'ensemble d'apprentissage :

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} f(\theta; x^{(i)}, y^{(i)})$$

- L'ensemble d'apprentissage doit être mélangé aléatoirement
- On peut faire plusieurs passages sur l'ensemble d'apprentissage avec des mélanges différents. Un passage complet s'appelle une **epoch**
- En pratique, le taux d'apprentissage doit s'adapter (cf. cours 6)

En deep learning, on utilise un compromis appelé *minibatch*, qui utilise plusieurs exemples à chaque itération :

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} f(\theta; x^{(i:i+B)}, y^{(i:i+B)}) = \theta - \frac{\epsilon}{B} \nabla_{\theta} \sum_{i=i_0}^{i_0+B} L(x^{(i)}, y^{(i)}, \theta)$$

Gradient stochastique (intro)

Rappel :

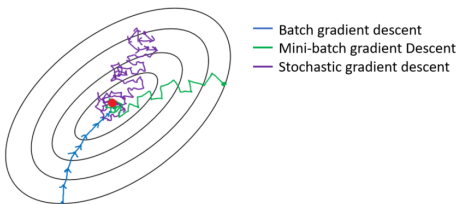
- perceptron « en ligne » $\Leftrightarrow$ descente gradient stochastique
 - ▶ correction à chaque exemple d'apprentissage
- perceptron « batch » $\Leftrightarrow$ descente de gradient classique
 - ▶ correction selon tous les exemples d'apprentissage

Ainsi :

- le mode « minibatch » est intermédiaire
 - ▶ correction selon plusieurs exemples d'apprentissage
 - ▶ intéressant si le calcul de la correction peut être rapide (parallélisation)
 - ▶ limité par les capacités matérielles (mémoire GPU)
- En pratique, $B = 8 \rightarrow 256$
 - ▶ On peut « simuler » des gros minibatches sur plusieurs itérations réelles : peu utile pour le temps de calcul, mais estimation plus robuste (cf. régularisation)

Gradient stochastique (intro)

- Plus on utilise d'exemples, plus l'estimation du gradient est robuste
- Plus on utilise d'exemples, plus les calculs et l'occupation mémoire sont importants



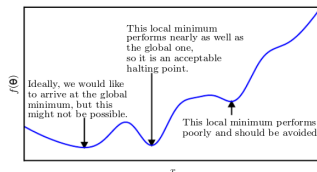
crédits : Imad Dabbura

Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût**
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion

Principe général

- design de l'architecture du réseau (→ section 4)
 - ▶ type de couches (→ **MLP**, CNN, RNN,...)
 - ▶ connexions entre couches
- **choisir une fonction de coût f**
- minimiser f par descente de gradient stochastique
 - ▶ utilisation d'un ensemble d'apprentissage
 - ▶ évaluation de l'erreur sur un ensemble de validation
 - calcul du gradient par rétro-propagation (→ section 5)
- on ne cherche pas systématiquement le minimum global
- réduire la fonction de coût (hors minima, même local) est déjà bien



Choix de la fonction de coût

- Cas général : principe du maximum de vraisemblance
- Très lié au *type de sorties* utilisé

Estimation du maximum de vraisemblance (*maximum likelihood*)

Soit :

- un ensemble de données $\{x_1, x_2, \dots, x_n\}$ provenant d'un tirage i.i.d de la distribution $p_{data}(x)$, considérée comme « vraie » mais « inconnue »
- la famille de modèles paramétriques $p_{model}(x; \theta)$. Elle donne notre estimation de la « vraie probabilité » $p_{data}(x)$

L'estimateur du maximum de vraisemblance (avec observations i.i.d) est :

$$\theta_{ML} = \arg \max_{\theta} \prod_{i=1}^n p_{model}(x_i; \theta) \quad (26)$$

Choix de la fonction de coût

- Pour des problèmes calculatoires (entre autres), on considère de préférence la log-vraisemblance :

$$\theta_{ML} = \arg \max_{\theta} \sum_{i=1}^n \log p_{model}(x^{(i)}; \theta) \quad (27)$$

- $\log$ étant croissante et continue, le maximum est le même.
- θ_{ML} coïncide avec le maximum a posteriori θ_{MAP} en supposant une distribution a priori *uniforme* des paramètres θ
- Peut être généralisé par l'estimateur du maximum de la (log)vraisemblance conditionnelle
 - ▶ la prédiction de labels sachant des données est le cas le plus usuel (appr. supervisé)

$$\theta_{ML} = \arg \max_{\theta} \sum_{i=1}^n \log p(y^{(i)} | x^{(i)}; \theta) \quad (28)$$

Dans un cas discret, on considère une fonction de masse $P()$ plutôt qu'une densité $p()$

Divergence de Kullback Leibler

distance (rappel)

Un ensemble est qualifié de métrique quand il est pourvu d'une fonction $d(x, y)$ à valeur dans $\mathbb{R}^+$ vérifiant :

- ① $d(x, y) = 0 \Rightarrow x = y$
- ② $x = y \Rightarrow d(x, y) = 0$
- ③ $d(x, y) = d(y, x)$ (symétrie)
- ④ $d(x, y) + d(y, z) \geq d(x, z)$ (inégalité triangulaire)

La fonction d est une distance ou une métrique.

- (1) et (2) constituent la propriété de *séparation*
- En l'absence de séparation (avec $d(x, x) = 0$) on parle de *pseudo-métrique*.
- Avec (2) et (3) seulement (avec $d(x, y) \in \mathbb{R}^+$), on parle de *dissimilarité*

Divergence de Kullback Leibler

Soient P et Q deux lois de probabilité admettant les densités p et q par rapport à une mesure de référence λ . L'information de Kullback ou entropie relative ou divergence de KL est :

$$KL(P, Q) = \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{x \sim p} \left(\log \frac{p(x)}{q(x)} \right) \quad (29)$$

Un symétrisé de cette grandeur est appelé divergence de Jeffreys(-Kullback-Leibler) :

Soient P et Q deux lois de probabilité admettant les densités p et q . Leur divergence de Jeffreys est :

$$J(P||Q) = KL(P, Q) + KL(Q, P) = \int_{-\infty}^{+\infty} (p(x) - q(x)) \log \frac{p(x)}{q(x)} dx \quad (30)$$

Divergence de Kullback Leibler

KL est *presque* une distance

La divergence de Kullback Leibler :

- est positive
- peut être symétrisée
- $KL(P, Q) = 0$ ssi $P = Q$ presque partout (Exercice : le prouver)

→ on a toutes les propriétés de la distance sauf inégalité triangulaire.

Il existe d'autres méthodes de symétrisation e.g Jensen Shannon :

$$JS(P||Q) = \frac{KL(P||\frac{P+Q}{2}) + KL(Q||\frac{P+Q}{2})}{2} \quad (31)$$

Sa racine carrée induit une métrique vérifiant l'inégalité triangulaire et :

$$0 \leq JS(P||Q) \leq \frac{1}{4} J(P||Q) \quad (32)$$

Choix de la fonction de coût

- Estimation du maximum de (log) vraisemblance :

$$\theta_{ML} = \arg \max_{\theta} \sum_{i=1}^n \log p_{model}(x_i; \theta) \quad (33)$$

$$= \arg \max_{\theta} \mathbb{E}_{x \sim \hat{p}_{data}} [\log p_{model}(x; \theta)] \quad (34)$$

- Interprétation : minimisation de $KL(\hat{p}_{data}, p_{model})$
où $\hat{p}_{data}$ est la distribution empirique induite par les $\{x_i\}$

$$KL(\hat{p}_{data}, p_{model}) = \mathbb{E}_{x \sim \hat{p}_{data}} [\log \hat{p}_{data}(x) - \log p_{model}(x)] \quad (35)$$

Comme $\log \hat{p}_{data}(x)$ est indépendant du modèle, cela revient à minimiser :

$$-\mathbb{E}_{x \sim \hat{p}_{data}} [\log p_{model}(x)] \quad (36)$$

C'est la **cross entropie** (entropie croisée?) de $\hat{p}_{data}$ et p_{model}

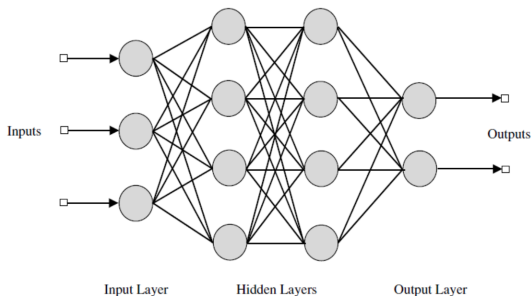
Choix de la fonction de coût : résumé

- Un réseau de neurones est un modèle paramétrique définissant une distribution $p(\mathbf{y}|\mathbf{x}; \theta)$
- L'application du maximum de vraisemblance donne la fonction de coût (à minimiser) suivante (*NLL : negative log likelihood*) :

$$J(\theta) = -\mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{data}} [\log p_{model}(\mathbf{y}|\mathbf{x})] \quad (37)$$

- $J(\theta)$ est la cross entropie de $\hat{p}_{data}$ et p_{model} et sa minimisation est équivalente à celle de $KL(\hat{p}_{data}, p_{model})$
- La forme exacte de $J(\theta)$ ne dépend que de p_{model} donc de notre modélisation de la tâche / du problème
- Cette forme dépend donc en particulier de la représentation des sorties
 - ▶ $\rightarrow$ binary-crossentropy, binomial-crossentropy...

Couche de sortie



- On suppose que le MLP transforme ses entrées en des caractéristiques $h = f(x; \theta)$ via ses couches cachées.
- On s'intéresse à la manière de produire les valeurs de sortie $\hat{y}$

Couche de sortie

Unité de sortie linéaire

Étant donné des caractéristiques h , une couche de sortie d'unités linéaires donne le vecteur $\hat{y} = \mathbf{W}^T \mathbf{h} + b$

- Modélisation d'un problème de **régression** (estimation grandeur continue non bornée)
- Classiquement cette couche modélise une dépendance gaussienne :
 $p(\mathbf{y}|\mathbf{x}) = \mathcal{N}(\mathbf{y}; \hat{\mathbf{y}}, \mathbf{I})$
 - ▶ $\hat{y}(\mathbf{x}; \mathbf{w})$ est une fonction estimant la moyenne de la Gaussienne.
- Maximiser la log-vraisemblance (pour trouver les paramètres $\mathbf{w}^*$ optimaux) est alors « équivalent » à minimiser l'erreur quadratique (MSE : *mean square error*) → régression linéaire
 - ▶ mêmes paramètres optimaux (ceux qui maximisent vraisemblance / minimisent MLE)
 - ▶ mais pas le même coût : le « chemin » pour y arriver est $\pm$ approprié

Pour une sortie, on définit :

$$p(y|x) = \mathcal{N}(y; \hat{y}, \sigma^2) \quad (38)$$

$$= \sqrt{\frac{1}{2\pi\sigma^2}} \exp\left(-\frac{(y - \hat{y})^2}{2\sigma^2}\right) \quad (39)$$

Si on a m échantillons tirés i.i.d, la log-vraisemblance conditionnelle est :

$$\sum_{i=1}^m \log p(y_i|x_i; \mathbf{w}) = -m \log \sigma - \frac{m}{2} \log(2\pi) - \sum_{i=1}^m \frac{|y_i - \hat{y}_i|^2}{2\sigma^2} \quad (40)$$

Ce qui donne bien le même maximum (i.e même poids optimaux $\mathbf{w}^*$ de $\hat{y}(\mathbf{x}; \mathbf{w})$) que $\frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|^2$

Pour n sorties, on définit :

$$p(y|x) = \mathcal{N}(y; \hat{y}, I) \quad (41)$$

$$= \sqrt{\frac{1}{(2\pi)^n}} \exp \left(-\frac{(y - \hat{y})^T (y - \hat{y})}{2} \right) \quad (42)$$

Si on a m échantillons tirés i.i.d, la log-vraisemblance conditionnelle est :

$$\sum_{i=1}^m \log p(y_i | x_i; \theta) = -\frac{mn}{2} \log(2\pi) - \sum_{i=1}^m \frac{\|y_i - \hat{y}_i\|^2}{2} \quad (43)$$

Et on retrouve aussi une minimisation MSE à θ_{MV} .

Couche de sortie

Il est courant de devoir prédire une variable y binaire :

- il faut déclencher l'alarme incendie (ou pas)
- il y a un obstacle devant la voiture (ou pas)
- il faut vendre l'action (ou l'acheter)
- il y a un visage dans cette (sous) image (ou pas)
- le locuteur est connu du système (ou pas)

Le réseau de neurone doit donc prédire $P(y = 1|x)$. Pour être dans $[0, 1]$ avec des unités linéaires on peut prendre :

$$P(y = 1|x) = \max\{0, \min\{1, \mathbf{w}^T \mathbf{h} + b\}\} \quad (44)$$

Mais si $\mathbf{w}^T \mathbf{h} + b \notin [0, 1]$ alors $\nabla_{\mathbf{w}} J(\theta)$ est nul $\rightarrow$ la descente de gradient s'arrête !

Couche de sortie sigmoïdale

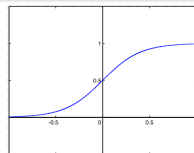
Unité de sortie sigmoïdale

Étant donné des caractéristiques $\mathbf{h}$, une couche de sortie sigmoïdale est définie par :

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{h} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{h} + b)}}$$

- Fonction sigmoïde

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Deux composantes :

- couche linéaire $z = \mathbf{w}^T \mathbf{h} + b$
- fonction d'activation sigmoid pour « transformer en probabilité »

Distribution de probabilité de la sigmoïde

Comment définir une probabilité y à partir de la composante linéaire z ?

- On considère une « probabilité non normalisée » $\tilde{P}$
- On suppose une dépendance linéaire de $\log \tilde{P}$ avec y et z

$$\log \tilde{P}(y) = yz \quad \text{donc} \quad \tilde{P}(y) = e^{yz}$$

En normalisant :

$$P(y) = \frac{e^{yz}}{e^{yz} + e^{(1-y)z}}$$

$$P(y) = \sigma((2y - 1)z)$$

- La variable z (score brut avant normalisation) définissant une distribution sur variables binaires est appelée *logit*.
- La normalisation conduit à une distribution de Bernoulli de paramètre « contrôlé par la sigmoïde » ($P(y = 1|x)$ vaut 1 avec proba $\sigma(\dots)$)

Couche de sortie sigmoïdale et MV

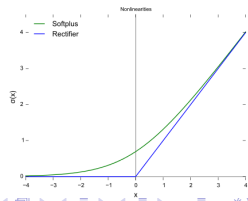
$$P(y) = \sigma((2y - 1)z) \quad (45)$$

La fonction de coût du maximum de vraisemblance pour une distribution de Bernoulli paramétrée par une sigmoïde est $-\log P(y|x)$ donc (avec $z = \theta^T \mathbf{x}$) :

$$\begin{aligned} J(\theta) &= -\log P(y|x) \\ &= -\log \sigma((2y - 1)z) \\ &= \zeta((1 - 2y)z) \end{aligned}$$

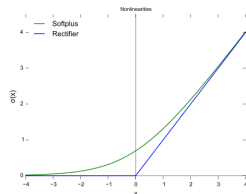
- **zeta** : Softplus (version « douce » de ReLU)

$$\zeta(x) = \ln(1 + e^{-x})$$



Couche de sortie sigmoïdale et MV

$$J(\theta) = \zeta((1 - 2y)z) \quad \text{et } z = \theta^T \mathbf{x}$$



- Le coût ζ sature uniquement quand $(1 - 2y)z \ll 0$ i.e :
 - $y = 1$ et $z \gg 0 \rightarrow$ « bonne réponse »
 - $y = 0$ et $z \ll 0 \rightarrow$ « bonne réponse »
- Quand z a le mauvais signe, alors $(1 - 2y)z \mapsto |z|$
 - Dans ce cas : $\lim_{|z| \rightarrow \infty} \zeta = |z|$
 - Donc

$$\frac{\partial \zeta}{\partial z} \mapsto \text{sign}(z)$$

$\rightarrow$ un z « très incorrect » ne modifie pas le (module du) gradient

$\rightarrow$ donc la descente de gradient peut immédiatement le (z) corriger!

Couche de sortie softmax

On s'intéresse couramment à des sorties donnant la distribution de probabilité d'une variable discrète avec n valeurs possibles :

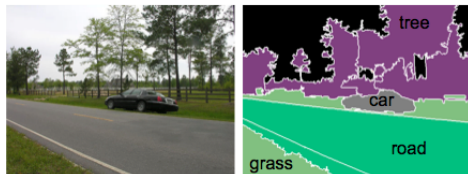
- reconnaissance d'une personne parmi n
- labellisation d'un pixel en végétation/route/bâtiment/véhicule...
- ...

C'est une généralisation du cas précédent (variable binaire). Nous souhaitons prédire $\hat{y}$ avec :

$$\hat{y}_i = P(y = i | \mathbf{x})$$

Contraintes

- comme avant : $\hat{y}_i \in [0, 1]$
- de plus : $\sum_i \hat{y}_i = 1$



Couche de sortie softmax

Comme précédemment, on a des scores bruts (log probabilités non normalisées) :

$$\mathbf{z} = \mathbf{W}^T \mathbf{h} + \mathbf{b} \text{ avec } z_i = \log \tilde{P}(y = i | \mathbf{x})$$

Unité de sortie softmax

Étant donné des caractéristiques h , une couche de sortie softmax est définie par :

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Couche de sortie softmax

L'application du maximum de (log)vraisemblance consiste à maximiser $\log P(y = i|x) = \log \text{softmax}(z)_i$, avec :

$$\log \text{softmax}(z)_i = z_i - \log \sum_j e^{z_j} \quad (46)$$

- le terme z_i ne sature pas, assurant un gradient non nul (hors optimum!)
- la maximisation de l'entrée i est favorisée par :
 - ▶ un z_i correct
 - ▶ un ensemble des autres z_j faibles
- → un exemple appartenant effectivement à la classe i doit maximiser la probabilité de la sortie i et minimiser les autres

Couche de sortie softmax

- Un score brut z_i donne une activité (sortie) $a_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$
 - Le plus grand z_i (*max*) est renforcé ; les autres z_i sont amortis
 - la sortie est une fonction « *max adoucies* » (→ **différentiable**)

- $\sum_i z_i = 3.5$

- $\sum_i a_i = 1$



$$z_1^L =$$

1.8



$$z_2^L =$$

-1



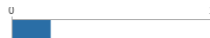
$$z_3^L =$$

3.2



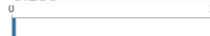
$$z_4^L =$$

0.5



$$a_1^L =$$

0.186



$$a_2^L =$$

0.011



$$a_3^L =$$

0.753



$$a_4^L =$$

0.051

Couche de sortie softmax

L'application du maximum de (log)vraisemblance consiste à maximiser $\log P(y = i|x) = \log \text{softmax}(z)_i$, avec :

$$\log \text{softmax}(z)_i = z_i - \log \sum_j e^{z_j} \quad (47)$$

- Notons que $\log \sum_j e^{z_j} \approx \max_j z_j$
(exp étant fortement convexe : $z_k < \max_j(z_j) \implies \exp(z_k) \ll \exp(\max_j(z_j))$)
- En cas de réponse (très) correcte pour l'entrée i , on a

$$\log \sum_j e^{z_j} \approx \max_j z_j = z_i$$

Donc dans ce cas $\log \text{softmax}(z)_i \approx \log \frac{z_i}{z_i} = 0$

→ cet exemple aura une faible contribution à la modification de l'entrée i (car elle est déjà correcte). Le coût total résulte alors principalement des autres exemples (encore) mal classifiés.

Bonus : sigmoïde vs softmax

- Pour $C=2$ classes, sorties sigmoïde et softmax sont équivalentes mais :
 - ▶ sigmoïde donne une prédiction $\hat{y}$ (pour l'autre classe : $1 - \hat{y}$)
 - ▶ softmax donne deux sorties (sommant à 1)
- Pour $C>2$, les deux sont utilisables :
 - ▶ utiliser softmax pour des problèmes **multiclass** (classes exclusives)
 - ▶ utiliser (multiples) sigmoïdes pour problèmes **multi-labels** (classes non exclusives)
 - ▶ les deux donnent C sorties a_j sur lesquelles calculer le coût.
- Expression des fonctions de coût :
 - ▶ (multi) sigmoïdes

$$J = -\frac{1}{n} \sum_x \sum_{j=1}^C (y_j \log a_j + (1 - y_j) \log(1 - a_j))$$

- ▶ softmax

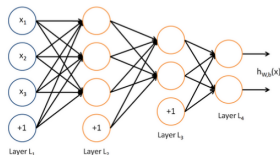
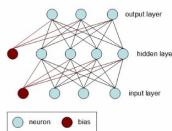
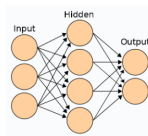
$$J = -\frac{1}{n} \sum_x \sum_{j=1}^C (y_j \log a_j)$$

Bonus : clarification de vocabulaire

- nombreuses expressions pour les *losses* dans les articles (forums, blogs...)
- max. de vraisemblance $\rightarrow$ en fait toujours **cross-entropy loss**
 - ▶ negative log-likelihood, logistic loss, multinomial logistic Loss
 - ▶ MultinomialLogisticLoss (caffe) BCELoss/NLLLoss (PyTorch) log_loss (tensorflow)
- softmax + CE loss = **categorical cross-entropy loss**
 - ▶ softmax loss
 - ▶ SoftmaxWithLoss Layer (caffe), CrossEntropyLoss (PyTorch), softmax_cross_entropy (tensorflow), [Sparse]CategoricalCrossentropy (tf.keras)
- sigmoid(s) + CE loss = **binary cross-entropy Loss**
 - ▶ Sigmoid Cross-Entropy loss
 - ▶ Sigmoid Cross-Entropy Loss Layer (caffe), BCEWithLogitsLoss (PyTorch), sigmoid_cross_entropy (tensorflow), BinaryCrossentropy (tf.keras)

source : Raúl Gómez Bruballa

Couches cachées et non linéarités



- Le design des couches cachées est un domaine de recherche actif
- → question encore relativement ouverte
 - ▶ ReLU ($g(z) = \max\{0, z\}$) est souvent un très bon choix
 - ▶ hyperparamètre pouvant être déterminé par validation croisée

Couches cachées et non linéarités

- Une couche cachée réalise généralement les opérations suivantes :
 - ▶ prend une entrée $\mathbf{x}$
 - ▶ calcule la transformation affine $\mathbf{z} = \mathbf{W}^T \mathbf{x} + b$
 - ▶ applique une fonction d'activation non linéaire $g(\mathbf{z})$ (par composante)
- Elles se distinguent essentiellement par la non-linéarité :
 - ▶ différentiables (sigmoïde, tanh)
 - ▶ différentiable presque partout (ReLU)
- En pratique, les logiciels retournent la dérivée à droite/gauche
 - ▶ l'optimisation est sujette à approximations de calcul
 - ▶ si on demande $g(0)$, il est probable que le logiciel retourne $g(\epsilon \sim 0)$

Couche cachée ReLU

- Fonction rectifieur linéaire (ReLU)

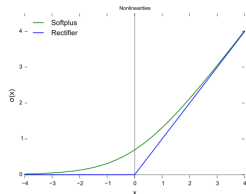
$$g(z) = \max(0, z)$$

- facile à optimiser car quasi linéaire
- dérivée 1 quand unité active $\rightarrow$ gradient grand et significatif
- dérivée seconde nulle $\rightarrow$ pas d'effet de second ordre sur la direction du gradient

En pratique, on calcule :

$$\mathbf{h} = g(\mathbf{W}^T \mathbf{x} + b)$$

Initialiser b à une petite valeur positive (0.1) assure que ReLU « fait passer le gradient » dès l'initialisation



Couche cachée ReLU

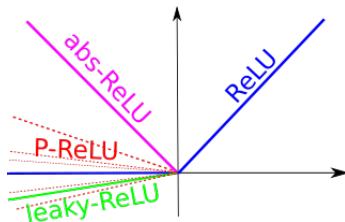
Il existe plusieurs généralisations de ReLU :

- performances comparables ou meilleures en pratique
- permet l'apprentissage sur les exemples d'activation nulle
- permet de « recevoir du gradient partout »

Définissent une pente α_i non nulle quand $z_i < 0$.

$$h_i = g(z, \alpha)_i = \max\{0, z_i\} + \alpha_i \min\{0, z_i\}$$

- rectification en valeur absolue
 - ▶ $\alpha_i = -1$ donc $g(z) = |z|$
 - ▶ fait sens pour trouver pattern inverse (image)
- ReLU avec fuite (*leaky ReLU*)
 - ▶ petit α_i (e.g 0.01)
- PReLU (*parametric ReLU*)
 - ▶ α_i est un paramètre supplémentaire à apprendre
 - ▶ possible activation négative $\Rightarrow$ accroît gradient de réseaux très profonds



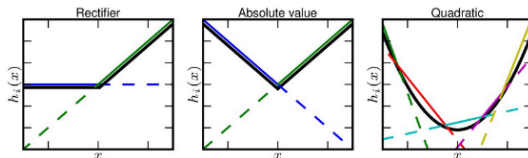
Couche cachée ReLU : maxout

Maxout [Goodfellow et al., ICML 2013] est une généralisation plus complexe.

- On a m unités qui renvoient le *max* de k combinaisons linéaires
- Pour une entrée $\mathbf{x} \in \mathbb{R}^d$, on a :

$$h_i(\mathbf{x}) = \max_{j \in G(i)} \left(\mathbf{x}^T W_{:ij} + b_{ij} \right) \quad (48)$$

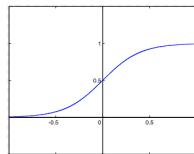
- où $G(i)$ est l'ensemble des indices du groupe $i \in [1, m]$
- On apprend les paramètres $W \in \mathbb{R}^{d \times m \times k}$ et $\mathbf{b} \in \mathbb{R}^{m \times k}$
- Une unité maxout apprend une fonction linéaire par morceaux (k morceaux) pouvant approximer toute fonction convexe :



Couches cachées sigmoïde et tanh

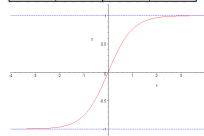
- Fonction sigmoïde

$$f(x) = \frac{1}{1+e^{-\lambda x}}$$



- Fonction tangente hyperbolique

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- fonctions d'activation les plus populaires avant l'introduction de ReLU
- utilisée pour prédire une probabilité (cf. avant)
- elles sont liées :

$$\tanh(z) = 2\sigma(2z) - 1$$

Couches cachées sigmoïde et tanh

- les sorties sigmoïdales saturent
 - ▶ à 1 quand $z \gg 0$
 - ▶ à 0 quand $z \ll 0$
- très sensible à l'entrée quand $z \approx 0$
- la saturation rend la descente de gradient difficile
→ déconseillé pour apprendre réseaux sauf si la fonction de coût « ôte » cette saturation
- *tanh* donne généralement de meilleurs résultats que σ
 - ▶ $\tanh(0) = 0$ versus $\sigma(0) = 1/2$
 - ▶ $\tanh(z) \sim z$ autour de 0
 - ▶ Ainsi *tanh* quasi équivalent à un modèle linéaire autour de 0
- $\sigma()$ a d'autres avantages pour les réseaux récurrents, autoencoders et autres modèles...

Autres couches cachées

Presque toute fonction différentiable peut être utilisée. Ex :
Identité :

- on remplace couche W par deux couches U et V , la couche U n'ayant pas de non linéarité (i.e identité)
- $h = g(W^T x + b)$ devient $h = g(V^T U^T x + b)$: factorisation de W
- diminue le nombre de paramètres $np \rightarrow (n + p)q$ (q est le nb sortie de U)
- acceptable si faible rang (i.e q petit)

RBF (radial basis function) :

- $h_i = \exp\left(-\frac{1}{\sigma^2} \|W_{:,i} - x\|^2\right)$
- forte réponse quand x est proche du « template » W
- saturation dans tous les autres cas $\rightarrow$ difficile à optimiser

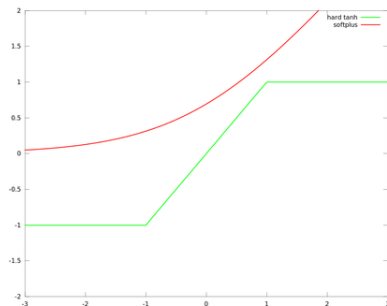
Autres couches cachées

Softplus :

- $\zeta(z) = \log(1 + e^z)$: version « douce » de ReLU
- meilleur en théorie car différentiable partout
- fonctionne moins bien en pratique

tanh dur (*hard tanh*)

- $g(z) = \max(-1, \min(1, z))$
- linéaire dans $[-1, 1]$ et bornée au delà

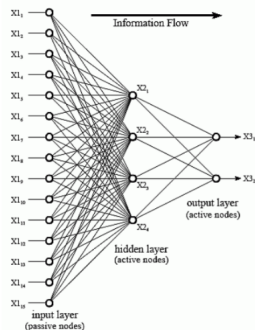


Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux**
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion

Architecture d'un réseau

- Architecture =
 - ▶ combien de neurones ?
 - ▶ comment les connecter ?
- Les neurones sont généralement organisés en **couches** (*layers*) :
 - ▶ combien de couches ?
 - profondeur du réseau (*depth*)
 - ▶ combien de neurones par couche ?
 - largeur du réseau (*width*)
 - ▶ quel type de couche ?
 - non linéarité
- L'architecture « idéale » dédiée à votre problème est généralement trouvée expérimentalement (train/val)
 - ▶ Partir d'architecture existantes est une bonne idée



Neural Network Zoo

Asimov Institute

Backfed Input Cell

Input Cell

Noisy Input Cell

Hidden Cell

Probabilistic Hidden Cell

Spiking Hidden Cell

Output Cell

Match Input Output Cell

Recurrent Cell

Memory Cell

Different Memory Cell

Kernel

Convolution or Pool

Perceptron (P)



Feed Forward (FF)



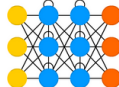
Radial Basis Network (RBF)



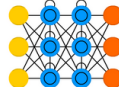
Deep Feed Forward (DFF)



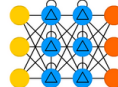
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)



Gated Recurrent Unit (GRU)



Auto Encoder (AE)



Variational AE (VAE)



Denoising AE (DAE)



Sparse AE (SAE)



Neural Network Zoo

Asimov Institute

Markov Chain (MC)



Hopfield Network (HN)



Boltzmann Machine (BM)



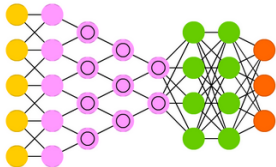
Restricted BM (RBM)



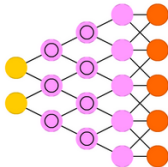
Deep Belief Network (DBN)



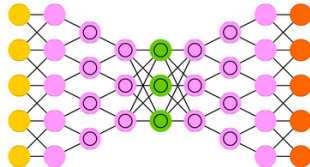
Deep Convolutional Network (DCN)



Deconvolutional Network (DN)



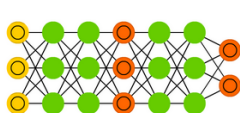
Deep Convolutional Inverse Graphics Network (DCIGN)



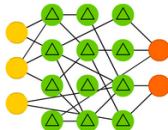
Neural Network Zoo

Asimov Institute

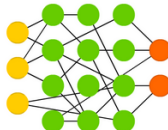
Generative Adversarial Network (GAN)



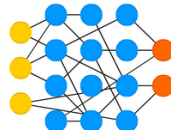
Liquid State Machine (LSM)



Extreme Learning Machine (ELM)



Echo State Network (ESN)



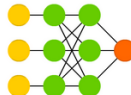
Deep Residual Network (DRN)



Kohonen Network (KN)



Support Vector Machine (SVM)



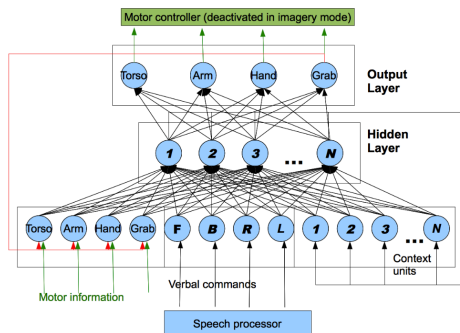
Neural Turing Machine (NTM)



Architectures spécifiques

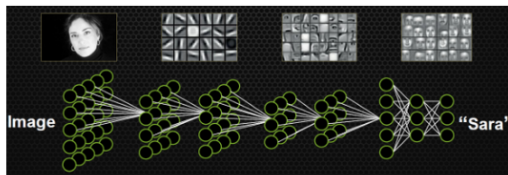
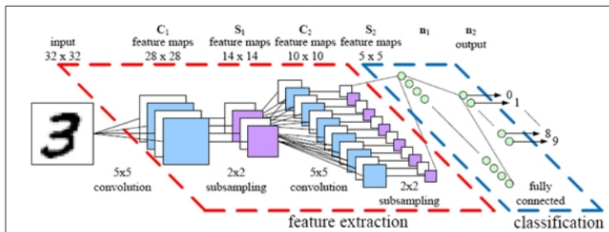
[Di Nuovo *et al.*, IJCNN 2012]

- comprendre le lien entre imagerie médicale et activité motrice
→ comment les « images mentales » influencent les mouvements et les capacités motrices ?
- mélange de plusieurs types d'entrées



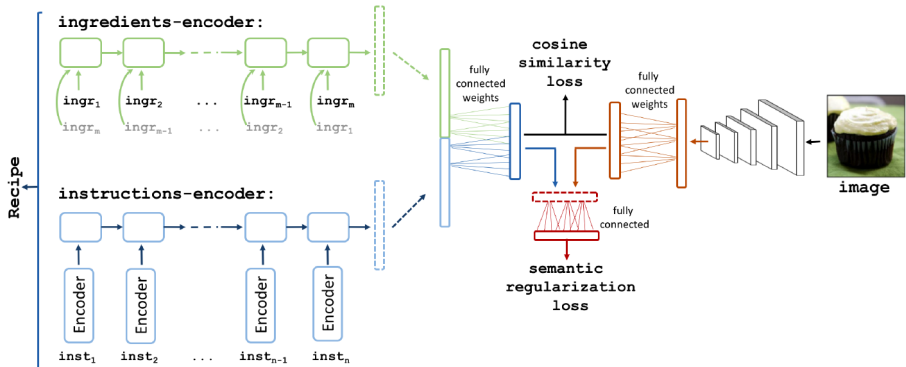
Réseaux convolutifs

(voir prochain cours pour les détails)



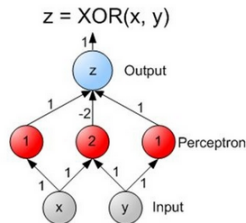
Associations d'architectures

- plusieurs type d'archi peuvent être associés
- en particulier pour traiter plusieurs média simultanément
 - ▶ Ex : [Salvador et al., 2017] utilise CNN, LSTM, BiLSTM (et MLP)



Théorème d'approximation universelle

- Un modèle « purement linéaire »
 - ▶ multiplication matricielle d'une couche à une autre
 - ▶ fonction de coût convexe → facile à optimiser



- Mais on veut parfois modéliser des problèmes (fortement) non linéaires
 - ▶ design d'une famille de fonction pour chaque non linéarité?
 - ▶ → un MLP avec 1 couche cachée et un nombre fini de neurones peut approximer toute fonction continue sur un compact de $\mathbb{R}^n$!
 - ★ des réseaux simples *peuvent représenter* de nombreuses fonctions si on a les paramètres appropriés
 - ★ **mais** il n'est pas garanti de pouvoir trouver ces paramètres algorithmiquement
 - ★ même pas par apprentissage !

Théorème d'approximation universelle

Théorème d'approximation universelle

Soit $\phi()$ une fonction continue strictement croissante et bornée. Pour toute fonction $f \in \mathcal{C}^0([0, 1]^m)$ et $\epsilon > 0$, il existe $N \in \mathbb{N}$ et pour $i = 1, \dots, N$, $v_i, b_i \in \mathbb{R}$ et $w_i \in \mathbb{R}^m$ tel que on puisse définir :

$$F(x) = \sum_{i=1}^N v_i \phi(w_i^T x + b_i)$$

qui approxime f à ϵ près : $\forall x \in [0, 1]^m \quad |F(x) - f(x)| < \epsilon$

Ainsi les fonctions de la forme $F(x)$ sont denses dans $\mathcal{C}^0([0, 1]^m)$

- $\phi()$ est une fonction d'activation

Théorème d'approximation universelle et NoFLT

- « $\forall f \in \mathcal{C}^0([0, 1]^m)$ » $\rightarrow$ on peut représenter toute fonction continue
- « $\exists N, v_i, b_i, w_i$ » $\rightarrow$ le théorème ne dit pas comment les trouver
 - ▶ l'optimisation peut ne pas trouver les paramètres
 - ▶ l'optimisation par apprentissage peut trouver de faux paramètres (sur-apprentissage)

Autre mauvaise nouvelle :

No free lunch theorem [Wolpert, 1956]

Plusieurs énoncés possibles (non formels) :

- Pour tout algorithme d'apprentissage A et B, il y a autant de problèmes où A a une meilleure erreur de généralisation que B que le contraire. En particulier, A peut être « choix aléatoire ».
- si on *moyenne* sur *tous* les problèmes d'apprentissage possibles, tous les algorithmes d'apprentissage ont la même erreur de généralisation
- aucun algorithme d'apprentissage est *universellement* meilleur qu'un autre

Théorème d'approximation universelle et NoFLT

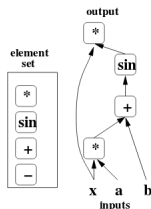
- Conséquence du NoFLT : il n'y a pas de procédure universelle pour déterminer une fonction généralisant *tout* ensemble d'apprentissage.
- Le théorème d'approximation universelle :
 - ▶ assure qu'il existe un NN assez grand pour approximer toute fonction continue
 - ▶ ne donne pas la taille correspondante
- il existe des bornes théoriques sur cette taille pour un NN à 1 couche approxinant une classe assez large de fonctions [Barron, 1993]
 - ▶ malheureusement, le pire cas nécessite un nombre exponentiel d'unités cachées

Un MLP à 1 couche peut approximer toute fonction, mais la couche peut être « infiniment » large. Des réseaux profonds peuvent :

- réduire le nombre de paramètres
- réduire l'erreur de généralisation

Profondeur de réseau

- un graphe de flot est un graphe qui représente un calcul
- chaque nœud représente une opération élémentaire et une valeur (le résultat de cette opération sur les enfants de ce nœud)
- Ex : $x.\sin(ax + b)$



- **Profondeur** (*depth*) : la longueur du plus long chemin depuis une entrée jusqu'à une sortie.
- Pour un MLP : nombre de couches cachées + 1 couche de sortie.

Intérêt de la profondeur

Théorème de la profondeur

[Montufar et al., 2014] Le nombre de régions (« polygonales ») générées par un réseau à activations ReLU à d entrées, n unités par couche cachée et de profondeur ℓ est :

$$O\left(\binom{n}{d}^{d(\ell-1)} n^d\right) \quad (49)$$

Pour des sorties maxout à k régions, le nombre de régions est :

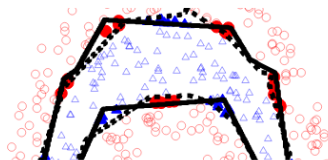
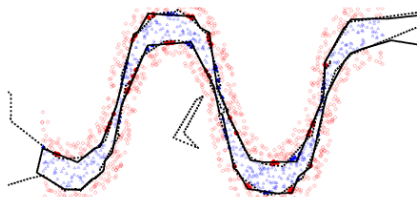
$$O\left(k^{(\ell-1)+d}\right) \quad (50)$$

- le nombre de régions différentes croît exponentiellement avec la profondeur !

Intérêt de la profondeur

[Montufar et al., 2014]

- MLP à 1 couche de 20 unités (ligne pleine) *versus* MLP à 2 couches de 10 unités (pointillés) :

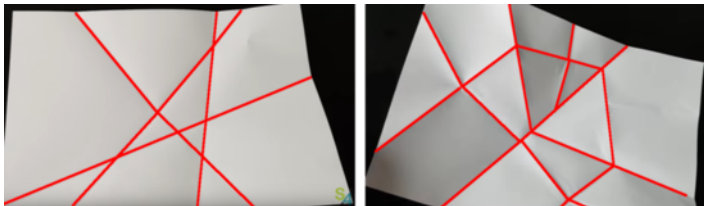


Intérêt de la profondeur

Théorème de la profondeur : nb régions définies par un NN

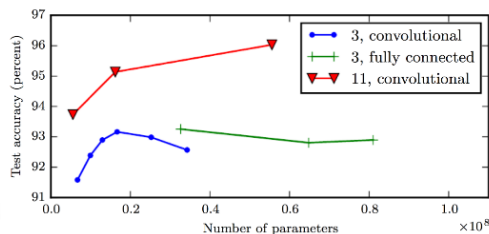
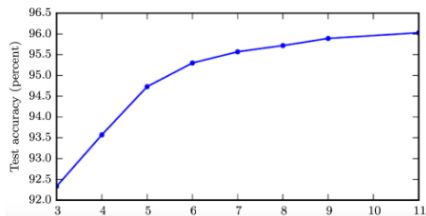
[Raghu et al., 2017] Pour un MLP à n couches cachées de largeur k et des entrées dans $\mathbb{R}^m$ le nombre de « régions linéaires » définies est majoré par $O(k^{mn})$ pour des activations ReLU et $O((2k)^{mn})$ pour des activations *hard tanh*.

- Généralisé aux activations *sigmoid*
- Illustration par des pliages de feuille (vidéo Science4all)
 - ▶ largeur : régions $\sim n^2/2$; profondeur : régions $\sim 2^n$



Intérêt de la profondeur

- (gauche) en pratique, la profondeur améliore les résultats (axe X = nb couches cachées)
- (droite) augmenter le nombre de neurones (paramètres) sans augmenter la profondeur est nuisible
→ importance du choix de l'architecture



[Goodfellow *et al.*, 2014]

Architecture : remarques finales

- Il existe des architectures plus spécialisées (préférentielles) selon les domaines : (à nuancer)
 - ▶ CNN pour la vision
 - ▶ RNN (et LSTM) pour le langage, la parole et les séquences
- il n'est pas obligatoire de connecter complètement la partie linéaire (→ CNN...)
- des architectures récentes (ResNet, DenseNet...) connectent les couches i aux couches $i + (k > 1)$
- Sous-tend des croyances sur la fonction $F(\mathbf{x}, \theta)$ estimée
 - ▶ c'est une composition de fonction simples

$$F(\mathbf{x}; \theta) = f_{out} \circ g_L \circ f_L \circ \dots \circ g_1 \circ f_1(\mathbf{x})$$
 (f_i = linéaire et g_i = activation non-linéaire)
 - ▶ elle résulte de variations successives « plus simples »
 - ▶ c'est un « programme informatique » d'étape successives, utilisant la sortie précédente
 - ★ certaines sorties intermédiaires peuvent être des « variables internes » aidant le calcul global

Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient**
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion

Rétro-propagation

- Rappel : l'apprentissage d'un NN est basé sur la minimisation d'une fonction de coût $J(\theta)$
- En pratique, cette minimisation est réalisée par descente de gradient (stochastique) : adapter itérativement les paramètres (poids du réseau) de manière à diminuer $J(\theta)$

$$\mathbf{w} \leftarrow \mathbf{w} - \epsilon \nabla_{\mathbf{w}} J(\theta) \quad (51)$$

- Il faut donc pouvoir calculer le gradient $\nabla_{\mathbf{w}} J(\theta)$
 - ▶ très facile à exprimer analytiquement
 - ▶ très coûteux à calculer
- la **rétro-propagation** est une technique efficace de calcul du gradient
 - ▶ ce n'est **pas** une méthode d'apprentissage
 - ▶ ce n'est **pas** propre aux réseaux de neurones

Dérivée de fonction composée

- La dérivée de fonction composée est appelée **chain rule of calculus**
il y a aussi une *chain rule* (différente) en probabilités

- Si $y = g(x)$ et $z = f(y) = f(g(x)) = (f \circ g)(x)$:

$$\frac{df}{dx} = f'(g(x))g'(x) \quad \text{ou} \quad \frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} \quad (52)$$

- cas vectoriel : $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, $g : \mathbb{R}^m \mapsto \mathbb{R}^n$ et $f : \mathbb{R}^n \mapsto \mathbb{R}$:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i} \quad (53)$$

En notation vectorielle :

$$\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^T \nabla_{\mathbf{y}} z \quad (54)$$

où $\left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)$ est la jacobienne ($n \times m$) de $g()$

Dérivée de fonction composée sur tenseurs

- Backprop = suite de multiplications « jacobienne $\times$ gradient »
- En pratique, on manipule des tenseurs (e.g 3D, 4D)_(pour minibatch notamment)
- Conceptuellement, cela revient à « réordonner » le tenseur en vecteur et à appliquer le cas précédent
- le gradient de z par rapport au tenseur $\mathbf{X}$ est noté $\nabla_{\mathbf{X}} z$
- les coordonnées de $\mathbf{X}$ sont des t-uples i (e.g = $\{i_1, i_2, i_3\}$ en 3D)
- Ainsi $(\nabla_{\mathbf{X}} z)_i = \frac{\partial z}{\partial \mathbf{X}_i}$
- Avec $\mathbf{Y} = g(\mathbf{X})$ et $z = f(\mathbf{Y})$:

$$\nabla_{\mathbf{X}} z = \sum_j (\nabla_{\mathbf{X}} \mathbf{Y}_j) \frac{\partial z}{\partial \mathbf{Y}_j} \quad (55)$$

- $\rightarrow$ même expression que précédemment !

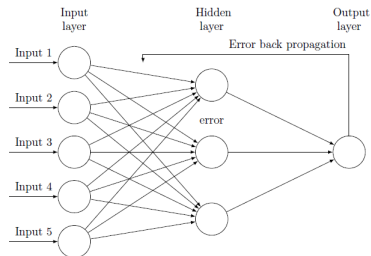
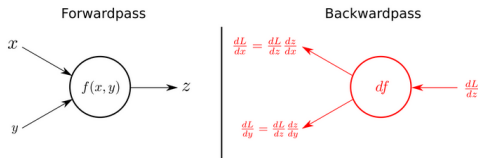
Rétro-propagation

- La rétro-propagation consiste en une application récursive (depuis le coût) de la *chain rule*
 - la passe directe permet de calculer la sortie estimée
 - la fonction de coût permet d'évaluer l'erreur avec la sortie attendue
 - la rétro-propagation permet d'évaluer le(s) gradient(s) et de le(s) propager en arrière pour mettre à jour les paramètres

$$\nabla_{\text{hidden out}} \rightarrow \text{m.a.j } w_{\text{hidden}}$$

$$\nabla_{\text{in out}} = \left(\frac{\partial \text{hidden}}{\partial \text{in}} \right)^T \nabla_{\text{hidden out}}$$

$$\rightarrow \text{m.a.j } w_{\text{in}}$$



Passe directe

- un réseau direct prend une entrée $\mathbf{x}$ et produit une sortie $\hat{\mathbf{y}}$
- l'information se propage de couche en couche, c'est la propagation directe (*forward propagation*)

$$\mathbf{h}^{(1)} = g^{(1)}(\mathbf{W}^{(1)}\mathbf{x}^{(1)} + \mathbf{b}^{(1)})$$

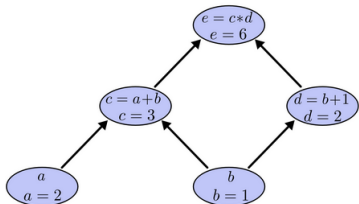
$$\mathbf{h}^{(2)} = g^{(2)}(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

...

$$\hat{\mathbf{y}} = g^{(d)}(\mathbf{W}^{(d)}\mathbf{h}^{(d-1)} + \mathbf{b}^{(d)})$$

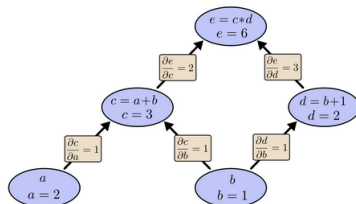
- Exemple pour $e = (a + b)(b + 1)$

source : Colah's blog



Dérivée dans les graphes de calcul

- les dérivées des nœuds connectés sont calculées « au niveau des arêtes »
- pour nœuds non connectés :
 - produit des arêtes formant un chemin entre les deux nœuds
 - sommation de tous les chemins



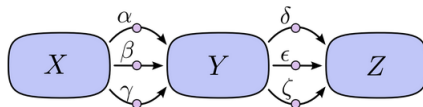
C'est une application de la *chain rule* (Eq. 53) :

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} = 2 \times 1 + 3 \times 1 = 5 \quad (56)$$

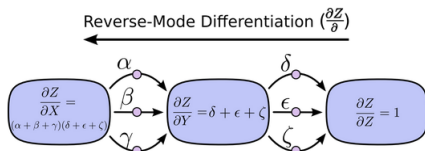
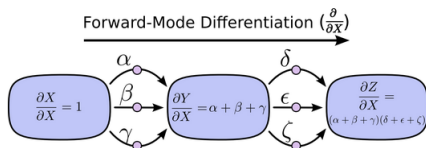
Dérivée dans les graphes de calcul

- Problème : la « sommation sur tous les chemins » peut devenir combinatoirement complexe

► $\frac{\partial Z}{\partial X} = \alpha\delta + \alpha\epsilon + \alpha\zeta + \beta\delta + \beta\epsilon + \beta\zeta + \gamma\delta + \gamma\epsilon + \gamma\zeta$

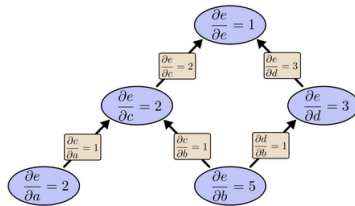
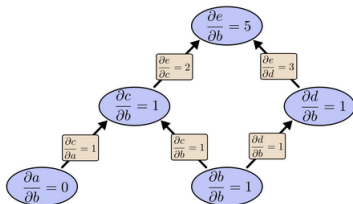


- Simplification par factorisation : $\frac{\partial Z}{\partial X} = (\alpha + \beta + \gamma)(\delta + \epsilon + \zeta)$
 ► deux « modes », faisant une seule passe par chaque arête :



Intérêt de la rétro-propagation

- Quel « mode » de différentiation utiliser ?
 - ▶ Exemple du calcul du $\frac{\partial e}{\partial b}$



- Le « mode reverse » (droite) permet d'obtenir $\frac{\partial e}{\partial a}$ directement !
 - ▶ Le « mode forward » (gauche) nécessite de refaire le calcul
- Gain $\times 2$ pour ce réseau... Mais bien plus grand avec un vrai NN !
 - ▶ Si on voulait la dérivée de nombreuses sorties pour un faible nombre d'entrées, le « mode forward » serait meilleur

Rétro-propagation MLP : algorithme direct

Considérons un MLP de profondeur l :

- avec des poids $\mathbf{W}_i$ et des biais $\mathbf{b}_i$ pour $i \in \{1, \dots, l\}$
- pour un échantillon $\mathbf{x}$, une sortie attendue y , un coût $L(y, \hat{y})$
- les pré-activation sont $\mathbf{a}_i$ et les couches après non linéarité f sont $\mathbf{h}_i = f(\mathbf{a}_i)$

① $\mathbf{h}_0 = \mathbf{x}$

② pour $k = 1$ à l faire :

$$\mathbf{a}_k = \mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k$$

$$\mathbf{h}_k = f(\mathbf{a}_k)$$

③ $\hat{y} = \mathbf{h}_l$

④ $J = L(y, \hat{y}) + \lambda \Omega(\mathbf{W}_i, \mathbf{b}_i)$

$\Omega()$ étant un terme de régularisation.

Rétro-propagation MLP : algorithme simple

(même réseau) on part de la couche de sortie pour calculer le gradient :

$$\textcircled{1} \mathbf{g} \leftarrow \nabla_{\hat{y}} J = \nabla_{\hat{y}} L(y, \hat{y})$$

Itérativement (en arrière) :

$$\textcircled{2} \text{ pour } k = l, l-1, \dots, 1 \text{ faire :}$$

Conversion du gradient de sortie en gradient de pré-activation par application de la *chain rule* ($\odot$ = multiplication « par élément »)

$$\text{i } g \leftarrow \nabla_{\mathbf{a}_k} J = \mathbf{g} \odot f'(\mathbf{a}_k)$$

Calcul du gradient sur les paramètres (avec la régularisation éventuelle)

$$\text{ii } \nabla_{\mathbf{b}_k} J = (\nabla_{\mathbf{a}_k} J) \frac{\partial \mathbf{a}_k}{\partial \mathbf{b}_k} = \mathbf{g} (+\lambda \nabla_{\mathbf{b}_k} \Omega(\dots))$$

$$\text{iii } \nabla_{\mathbf{W}_k} J = (\nabla_{\mathbf{a}_k} J) \frac{\partial \mathbf{a}_k}{\partial \mathbf{W}_k} = \mathbf{g} \mathbf{h}_{k-1}^T (+\lambda \nabla_{\mathbf{W}_k} \Omega(\dots))$$

Les paramètres peuvent être mis à jour ici.

Propagation du gradient à la couche suivante (en arrière)

$$\text{iv } g \leftarrow \nabla_{\mathbf{h}_{k-1}} J = \frac{\partial \mathbf{a}_k}{\partial \mathbf{h}_{k-1}} (\nabla_{\mathbf{a}_k} J) = \mathbf{W}_k^T \mathbf{g}$$

Rétro-propagation générale

- Algorithmes présentés sont pour cas particulier
 - ▶ coût supervisé
 - ▶ une seule entrée (vectorielle) $\mathbf{x}$
 - ▶ une seule estimation (vectorielle) $\hat{\mathbf{y}}$
- La forme générale de Backprop repose sur les mêmes idées
- Pour calculer le gradient de z w.r.t un de ses ancêtres $\mathbf{x}$
 - ▶ $\frac{\partial z}{\partial z} = 1$
 - ▶ gradient pour chaque parent de z obtenu par multiplication du gradient courant par la Jacobienne de la fonction produisant z
 - ▶ on continue ainsi (multiplications de Jacobiennes) en remontant le graph jusqu'à $\mathbf{x}$
 - ▶ Si il y a plusieurs chemins entre z et $\mathbf{x}$, on somme les contributions

Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)**
- 7 Conclusion

Modèle statistique de langage

- On considère une séquence de mots $w_i^j = (w_i, w_{i+1}, \dots, w_{j-1}, w_j)$
- Modèle de langage stat. = proba conditionnelle du prochain mot sachant tous les précédents

$$P(w_1^T) = \prod_{t=1}^T P(w_t | w_1^{t-1}) \quad (57)$$

- Utile pour *speech2text*, traduction, recherche d'information...
- Le modèle *n-gram* approxime en utilisant *n* précédents mots :

$$P(w_1^T) \approx \prod_{t=1}^T P(w_t | w_{t-n+1}^{t-1}) \quad (58)$$

- Reflète la probabilité d'appartenance conjointe et ordonnée de mots

Modèle statistique de langage

- Exemple : trigram (3-gram) :

$$\begin{aligned}
 P & \left(\text{Longtemps, je me suis couché de bonne heure.} \right) \\
 = & P(\text{heure}|\text{de, bonne})P(\text{bonne}|\text{couché, de})\dots \\
 & \dots P(\text{je}|\text{Longtemps, } < s >)P(\text{Longtemps} | < s >, < s >)
 \end{aligned}$$

- N-grams estimés par décomptage (fréq. d'apparition) dans un corpus
- Limites
 - ▶ n-grams rares quand le vocabulaire croît : certains n'apparaissent pas dans le corpus *train* : représentation parcimonieuse (*sparse*)
 - ▶ contexte limité (rarement plus de 4 mots)
 - ▶ ne modélise pas des « similarités » (sémantique, syntaxique...) entre mots

The cat is walking in the bedroom
A dog was running in a room

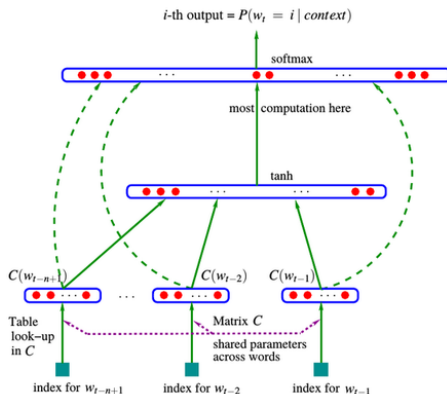
Modèle neuronal de langage

- Modèle neuronal de langage [Bengio et al., 2003]
 - ▶ représentation dense ($\in \mathbb{R}^m$) de chaque mot
 - ▶ représenter la proba jointe de la séquence de mots par une séquence de représentation de mots (concat. des repr. de mots)
 - ▶ apprentissage simultané des deux (repr. mots + proba jointe seq.)
- Des mots de contexte proche doivent avoir une représentation proche
- On apprend un modèle paramétrique f t.q :

$$\begin{aligned}
 f(w_t, \dots, w_{t-n+1}) &= P(w_t | w_1^{t-1}) \\
 f(i, w_{t-1}, \dots, w_{t-n+1}) &= g(i, C(w_{t-1}), \dots, C(w_{t-n+1}))
 \end{aligned}
 \tag{59}$$

- Matrice C : mapping des $|V|$ mots dans $\mathbb{R}^m$ partagé sur tous les mots
- $g(.)$ estime $P(w_t = i | w_1^{t-1})$

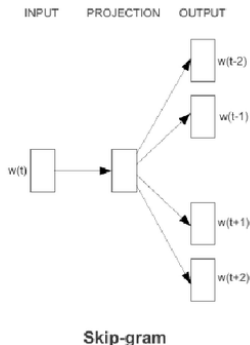
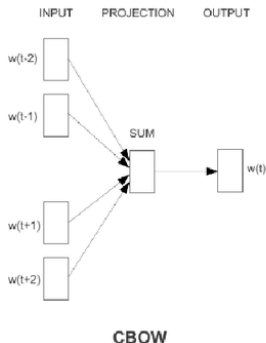
Modèle neuronal de langage



- Apprentissage par SGD de la log-vraisemblance (régularisée)
- Implémentation parallèle (3 semaines 40 CPU... en 2000)

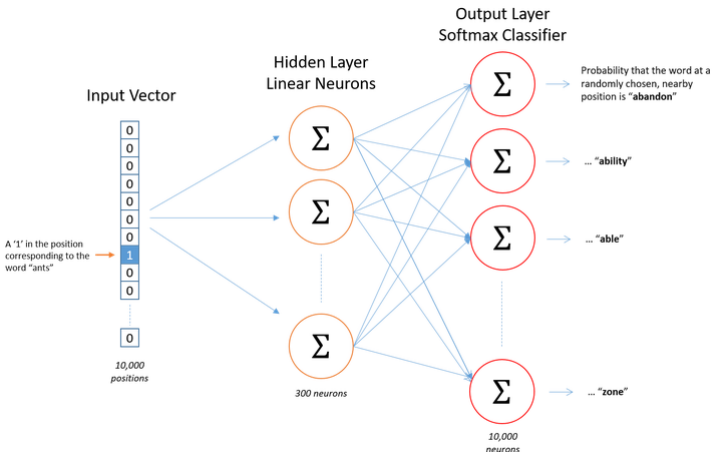
Modèle neuronal de langage : word2vec

- [Mikolov *et al.*, 2013] améliore l'efficacité d'apprentissage → word2vec
 - ▶ Continuous BoW (CBOW) : estime mot sachant contexte
 - ▶ Skip-gram : estime contexte sachant le mot (central)



Modèle neuronal de langage : word2vec

- Exemple de l'architecture skip-gram (NB : apprentissage **non** supervisé)

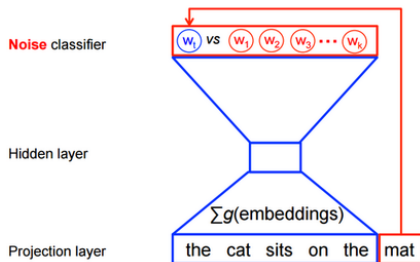


source : Chris McCormick

Modèle neuronal de langage : word2vec

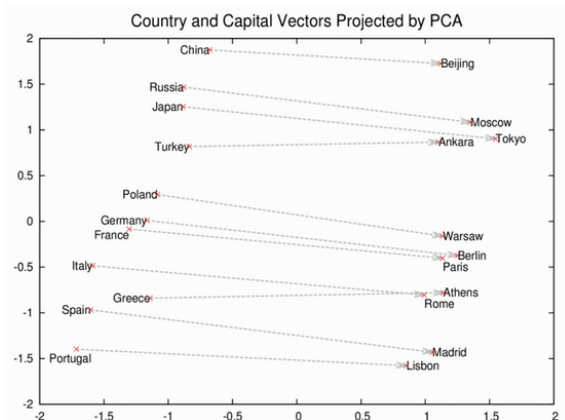
- Pour un mot w_t et son contexte C_t , softmax optimise

$$J = \sum_{t=1}^V \sum_{c \in C_t} \log p(w_c | w_t)$$
- Avec $V = 10k$, les calculs sont lourds !
- *Negative sampling* :
 - ▶ limiter le nombre d'échantillons mis à jour (1 pos + 4 neg)
 - ▶ classification binaire par classification logistique



Modèle neuronal de langage : word2vec

- word2vec : représentation vectorielle dense
 - arithmétique sémantique !



- couche cachée (e.g $D = 300$) sert de représentation pour NER, SRL, analyse sentiments, entity linking, QA...

Modèle neuronal de langage : FastText [Bojanowski et al., 2017]

- w2v représente les mots en fonction de leur *contexte*
 - ▶ ignore leur forme (morphologie)
 - ▶ ne gère pas les mots hors vocabulaire d'entraînement
- FastText applique w2v (skipgram) à des n -gram de caractères ($n = 3$)

Where $\rightarrow$ <where> $\rightarrow$ { <wh, whe, her, ere, re>, <where> }

- notons que *her* est différent de <*her*>

fastText

- en pratique,
 - ▶ tous les n -grams sont extraits pour $3 \leq n \leq 6$
 - ▶ chacun a une représentation z_g
 - ▶ un mot est représenté par la somme des z_g des n -gram qu'il contient
- Permet d'apprendre des représentations de mots rares
- Le code est disponible, avec des modèles

Plan

- 1 Introduction aux réseaux neuronaux profonds directs
- 2 Optimisation
- 3 Apprentissage d'un réseau et fonction de coût
- 4 Architecture des réseaux
- 5 Rétro-propagation du gradient
- 6 Représentation textuelle (peu profonde)
- 7 Conclusion**

Conclusion

- le perceptron multi couche (MLP) est un cas de :
 - ▶ réseau : composition de fonctions
 - ▶ neuronal : basé sur neurone formel
 - ▶ direct : sans boucle de retour
- apprend un mapping non linéaire entre features x et cibles y
- l'apprentissage se fait par optimisation (des paramètres/poids) d'une fonction de coût
 - ▶ descente de gradient : déplacement itératif dans la direction opposée au gradient
 - ▶ les valeurs propres de la matrice hessienne donnent la courbure de la fonction de coût : ils déterminent le taux d'apprentissage « optimal »
 - ▶ le gradient stochastique estime le gradient sur un ensemble réduit d'entrées (*minibatch*) et itère (plusieurs fois) sur l'ensemble d'apprentissage
 - ▶ Cf. cours 6!

Conclusion

- la fonction de coût résulte de l'application du principe du maximum de vraisemblance

$$J(\theta) = -\mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{data}} \log p_{model}(\mathbf{y}|\mathbf{x}) \quad (60)$$

- la forme exacte dépend du modèle paramétrique p_{model} choisi, en particulier du type des unités de sortie
- Les couches de sorties les plus communes sont :
 - ▶ linéaire
 - ▶ sigmoïde
 - ▶ softmax
- Le choix des non linéarités des couches cachées peut se faire par train/val
 - ▶ Mais ReLU est souvent un bon choix

Conclusion

- Il existe de nombreuses architectures de réseau
 - ▶ perceptron, MLP
 - ▶ récurrents : RNN, LSTM, GRU (Cf. cours 4)
 - ▶ non supervisées : AE, GAN (Cf. cours 7)
 - ▶ machines de Boltzman
 - ▶ convolutives (Cf. cours 3)
 - ▶ (...)
- En théorie, un MLP à une couche cachée peut approximer toute fonction continue
 - ▶ En pratique, rien ne dit qu'on puisse l'apprendre
 - ▶ Augmenter le nombre de couches est (donc) souvent bénéfique
- La rétro-propagation est une technique efficace de calcul du gradient
 - ▶ Consiste essentiellement en l'application récursive de la *chain rule*
- Modèles de langage neuronaux, dont *word2vec* et *FastText*

Références

- P Bojanowski, E Grave, A Joulin, T Mikolov (2017) Enriching Word Vectors with Subword Information, TACL(5) :135–146
- A. G. Di Nuovo, V. M. De La Cruz and S. Di Nuovo, (2012) Mental practice and verbal instructions execution : A cognitive robotics study. The 2012 International Joint Conference on Neural Networks (IJCNN), Brisbane, QLD, 2012, pp. 1-6.
- I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio (2013) Maxout networks, ICML 2013
- V. Mnih *et al.*, Human-level control through deep reinforcement learning, Nature 518, 529–533 (26 February 2015)
- Guido Montúfar, Razvan Pascanu, Kyunghyun Cho, Yoshua Bengio (2014) On the Number of Linear Regions of Deep Neural Networks. Advances in neural information processing systems, 2924-2932
- T Mikolov, I Sutskever, K Chen, GS Corrado, J Dean (2013) Distributed representations of words and phrases and their compositionality, NIPS
- M. Raghu, B. Poole, J. Kleinberg, S. Ganguli, J. Sohl-Dickstein (2017) On the Expressive Power of Deep Neural Networks, ICML
- Wolpert, David (1996) The Lack of A Priori Distinctions between Learning Algorithms, Neural Computation, pp. 1341-1390.